Lab 3

1. Write an assembly payroll program (which you should have from Lab 2) which will compute the total monthly pay of an employee, **in pennies**. The formula for total pay is as follows:

   Total monthly pay in $ = base pay in $ + number of overtime hours × (2.5 × hourly rate in $)

   In your code, you can assume that the monthly base pay is $5000, number of overtime hours in a month is 15, and hourly rate is $32.50. **You can only use integer arithmetic**. **No division operator is available. Your code must not use any loops or branches.**

   However, you can get very close to the exact salary (which is $6218.75). Here's your hint:

   $$\frac{1}{100} \approx \text{sum of several terms of the form } \frac{1}{2^n};\ \ n \text{ is an integer}$$

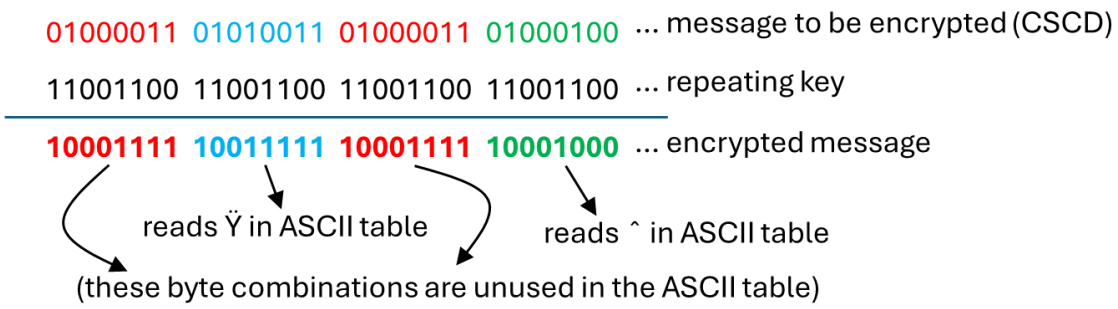
   See how close you can get to 0.01 (**but without exceeding it**) by following the recipe on the right-hand side of the above expression (e.g., $\frac{1}{32}+\frac{1}{64} \approx 0.05$), then implement your scheme. **Remember, your goal is to process the salary in pennies (621875) so that you can get close to the actual salary in dollars, $6218.75. Obviously, since you are dealing with integer arithmetic, the best you can hope for is $6218 (but you can't just move this number and be done with the code, you must follow the hint above and process the salary in pennies to get close to $6218)**. The number of points you score on this part will depend on how close your answer is to $6218.

2. Write an assembly program which will compute the first six Fibonacci numbers after 1 and 1. The Fibonacci sequence is defined as: $\text{Fib}(n) = \text{Fib}(n-1) + \text{Fib}(n-2)$, with the initial values $\text{Fib}(0) = \text{Fib}(1) = 1$. **Do not use a loop/branches**.

   Note: You can use at most three registers, R0, R1 and R2, and your six numbers should appear sequentially in R2. Seed R0 and R1 with 1 and 1 initially.

3. A very simple EOR/XOR based cryptographic algorithm works as follows. Suppose the message we want to encrypt is CSCD, which in ASCII is: 01000011 01010011 01000011 01000100. First, we choose a "byte key", e.g. 11001100. Next, we EOR the message we want to encrypt with the "repeating key" 11001100 11001100 11001100 11001100 to produce the encrypted message (see example below). The original message can be

recovered (decryption process) by EOR'ing the encrypted message with the "repeating key" (verify this for the encryption example shown below).



Write an assembly code which will perform the encryption and decryption processes described above. You can choose any 4-character word to be encrypted. The choice of the "byte key" is also up to you. **Do not use a loop/branches**.

Note: The word to be encrypted should be available in R0, the key should be available in R1, the encrypted word should be in R2, and the decrypted word should be in R3.